

SimPy KPI Lab 项目说明书

版本 0.2.0 (含 P0 轨迹导出与 mine/design/plot 扩展) | 项目目录: F:\Simpy\SimPy\SimPy\simpy-kpi-lab\simpy-kpi-lab | 更新日期: 2026-09-07

目录

1. 项目概述与设计理念
2. 目录结构
3. 快速开始
4. 环境变量与 API Key
5. 配置参考 (YAML)
6. 仿真模型与随机流
7. KPI 体系与统计口径
8. 输出文件
9. AI 分析层 (OpenAI / DeepSeek 双路径)
10. 安全闭环控制 API
11. MCP 接口
12. 事件轨迹记录与 XES 导出 (P0)
13. 数据采集与模型挖掘 (simlab mine)
14. LLM 蓝图式模型设计 (simlab design)
15. 结果可视化 (simlab plot)
16. 模块清单与职责
17. 扩展指南
18. 已知限制
19. 本机环境备忘

一、项目概述与设计理念

SimPy KPI Lab 是一个可复现、可扩展的离散事件仿真与安全决策框架：用 SimPy 描述排队系统，用多次 replication 与参数网格做实验，用 KPI 与置信区间比较场景，并可选择调用大模型（OpenAI 或 DeepSeek）生成结构化 KPI 分析、白名单动作提案或模型蓝图。

核心设计理念是 **“AI 永远只出提案/数据，绝不执行”**：模型输出必须经过本地 Pydantic 校验与白名单检查，参数调适需人工审批后由本地代码应用到新的不可变配置快照上，再跑下一轮仿真，形成闭环。AI 不参与 KPI 数值计算，不写代码，不接触文件系统。

真实事件日志 → (mine) 挖掘分布/容量/序列 → YAML 配置 —
自然语言描述 → (design) LLM 生成蓝图 → Pydantic 校验 → YAML 配置 —
↓
参数网格场景 → SimPy replications → KPI 统计与置信区间 → (plot) 图表报告
→ ① AI 结构化 KPI 分析 (ai.py)
→ ② AI 白名单动作提案 (agent.py) → 人工审批 (workflow.py) → 新配置版本 → 再仿真 (闭环)
→ ③ 事件轨迹 traces.xes / traces.jsonl (trace.py)

能力清单

- 串行多工位 SimPy 模型；每工位可配置容量与四种服务时间分布（指数 / 定值 / 均匀 / 三角）。
- 到达流与各工位使用独立随机流；默认共同随机数（CRN）；BLAKE2b 稳定派生 seed。
- 多次 replication、参数网格、多进程并行；KPI 输出均值、标准差、正态近似置信区间、最小最大值。
- JSON 与 CSV 输出；AI 分析层输出经 Pydantic 校验的 JSON 与 Markdown。
- AI 安全提案层：仅 set_parameter / change_policy / enable_resource 三类结构化动作 + 本地 allowlist 二次校验。
- 人工审批 workflow：五状态机、乐观版本号、幂等 operation ID、配置哈希、有序审计日志。
- FastAPI REST + WebSocket + 可选 Bearer Token；仿真与 AI 请求在线程池运行。
- MCP v2 受限客户端（不暴露建会话 / 运行 / 批准 / 拒绝）。
- 事件轨迹记录与 XES 导出（`experiment.record_traces`）。
- 数据驱动的模式挖掘（`simlab mine`）：从 XES/JSONL 日志估计工位序列、分布参数与并发容量。
- LLM 蓝图式模型设计（`simlab design`）：自然语言 → 校验过的仿真配置。
- 结果可视化（`simlab plot`）：KPI 对比、利用率、甘特图、模型结构图与 HTML 报告。
- OpenAI 与 DeepSeek 双提供商支持：自动按 base_url 切换 Responses / chat-completions 路径。

二、目录结构

```
simpy-kpi-lab/
├─ src/simlab/          # 包源码（17 个模块，见第十六节）
├─ tests/               # pytest 测试套件（95 个测试）
├─ examples/           # 演示配置与人工审批 PowerShell 脚本
├─ outputs/            # 实验输出（results.json / CSV / XES / 图表 / AI 分析）
├─ tools/              # Word 手册生成工具
├─ pyproject.toml       # 依赖与入口点（simlab / simlab-mcp; extras: dev/api/mcp/viz/docs）
├─ Dockerfile / Makefile
├─ .env.example        # 环境变量模板（密钥不入库）
├─ .env                # 本机实际密钥（已 gitignore，勿提交）
└─ closed_loop_demo.py  # 闭环演示驱动脚本
```

三、快速开始

要求 Python 3.11+（本机 F:\PY\Python 3.13.0）。Windows PowerShell:

```
py -3 -m venv .venv
.\.venv\Scripts\python.exe -m pip install -e ".[dev]"

.\.venv\Scripts\simlab.exe validate examples\service_center.yaml
.\.venv\Scripts\simlab.exe run examples\service_center.yaml --workers 4 --plot
.\.venv\Scripts\simlab.exe analyze outputs\service_center\results.json --question "比较各容量方案"
.\.venv\Scripts\simlab.exe mine outputs\service_center\traces.xes --output mined.yaml
.\.venv\Scripts\simlab.exe design "两工位服务中心：前台3分钟，专员4/8/15分钟，每小时12人" --output designed.yaml
.\.venv\Scripts\simlab.exe plot outputs\service_center\results.json
```

六个子命令：`validate`（校验配置）、`run`（执行实验，支持 `--workers/--output/--analyze/--plot/--question/--model`）、`analyze`（分析已有 results.json）、`plot`（生成图表）、`mine`（日志挖掘）、`design`（LLM 蓝图设计）、`serve`（启动闭环控制 API）。

四、环境变量与 API Key

变量	用途	必填
<code>OPENAI_API_KEY</code>	OpenAI 官方端点密钥（base_url 为空时读取）	用 OpenAI 时
<code>DEEPSEEK_API_KEY</code>	DeepSeek 端点密钥（base_url 含 deepseek 且未显式指定 api_key_env 时自动读取）	用 DeepSeek 时
<code>SIMLAB_OPENAI_BASE_URL</code>	切换 OpenAI 协议兼容端点（如 https://api.deepseek.com）；非空即走 chat-completions + JSON 模式	用 DeepSeek 时
<code>SIMLAB_OPENAI_MODEL</code>	覆盖 YAML 中的 openai.model（CLI 加载配置时生效）	否
<code>SIMLAB_API_TOKEN</code>	控制 API 管理员 Bearer Token；不设置则本机免认证	建议
<code>SIMLAB_API_OUTPUT_ROOT</code>	API 会话运行结果输出根目录（默认 outputs/api_sessions）	否
<code>SIMLAB_API_URL</code>	MCP 进程连接 REST 服务的地址（默认 http://127.0.0.1:8000）	MCP 时

安全约定：密钥只从环境变量 / .env 读取，绝不写入 YAML、results.json 或提交到版本库。 .env 已被 .gitignore 忽略。不带 --analyze 的仿真完全不需要密钥或网络。

五、配置参考 (YAML)

配置由 Pydantic 严格校验 (`extra="forbid"` , 拼错字段立即报错) 。最小配置:

```
simulation:
  until: 480                                # 仿真时长 (业务时间单位, 示例为分钟)
  warmup: 60                                # 预热期, 统计窗口为 [warmup, until)
  arrival_interarrival: {kind: exponential, mean: 5}
  stations:
    - name: service
      capacity: 2
      service_time: {kind: triangular, low: 4, mode: 8, high: 15}

experiment:
  replications: 20
  base_seed: 20260825
  common_random_numbers: true              # CRN: 场景间配对比较更精确
  confidence_level: 0.95
  parameter_grid:
    stations.0.capacity: [1, 2, 3]        # 点路径相对 simulation
  record_traces: false                     # true 时导出 traces.xes / traces.jsonl

openai:
  model: deepseek-chat                     # 或 gpt-4o-mini 等
  max_output_tokens: 2500
  timeout_seconds: 60
  max_retries: 2
  store: false
  base_url: https://api.deepseek.com       # 空 = OpenAI 官方端点
  api_key_env: DEEPSEEK_API_KEY           # 读取哪个环境变量
```

simulation 字段

字段	默认	说明
<code>name</code>	<code>service_system</code>	模型名
<code>until</code>	必填 >0	仿真结束时间
<code>warmup</code>	0	预热期 (必须小于 until)
<code>first_arrival_at_zero</code>	true	首个实体是否在 0 时刻到达
<code>max_arrivals</code>	null	最多到达实体数上限
<code>arrival_interarrival</code>	必填	到达间隔分布
<code>stations[]</code>	必填 ≥1	工位列表: name 唯一、capacity ≥1、service_time 分布
<code>cycle_time_target</code>	null	服务水平的周期时间目标

分布格式

```
{kind: exponential, mean: 5}      # 指数: mean > 0
{kind: deterministic, value: 5}   # 定值: value ≥ 0 (到达间隔要求 > 0)
{kind: uniform, low: 3, high: 7}  # 均匀: 0 ≤ low ≤ high
{kind: triangular, low: 3, mode: 5, high: 9}  # 三角: 0 ≤ low ≤ mode ≤ high
```

六、仿真模型与随机流

当前模型是“所有实体依次经过所有工位”的串行网络（`simulation.py` 的 `customer()` 生成器）：实体到达 → 逐工位排队（`simpy.Resource`）→ 服务（按分布采样时长）→ 完成。

随机流设计：到达流与每个工位的服务时间流相互独立，seed 由 `derive_seed(base_seed, namespace)` 用 BLAKE2b 稳定派生——工位随机流按名称派生，新增其他工位不会意外改变已有工位的随机序列。CRN 开启时，不同场景同编号 replication 使用相同 seed，适合容量方案的配对比较。

七、KPI 体系与统计口径

- **统计窗口**：[`warmup`, `until`]；周期时间只统计 warmup 后到达且结束前完成的 cohort 实体，避免预热期队列污染。
- **工位指标**：利用率 = 窗口内实际服务区间 / (capacity × 窗口长度)，跨边界区间自动裁剪；平均队长由排队时间面积计算，期末仍在排队的实体也计入。
- **服务水平** = 周期时间不超过 `cycle_time_target` 的 cohort 比例。
- **数据质量护栏**：`censored_cycle_count` 与 `cycle_completion_fraction` 显式显示右删失程度；`n_total` / `n_missing` 显示缺失值；只有一次有效 replication 时标准差与 CI 为 null，不伪装零不确定性。
- **角色标注**：每项指标带 role（primary / driver / guardrail / context / data_quality）、direction（越高/越低越好、目标区间）与单位，供 AI 分析层与下游消费方理解。

八、输出文件

```
outputs/service_center/
├─ results.json      # 配置、随机流信息、每次 replication、KPI 汇总
├─ replications.csv  # 每次实验的宽表 KPI
├─ summary.csv       # 每个场景/指标的统计汇总（均值、CI 等）
├─ traces.xes        # 事件轨迹（record_traces: true 时）
├─ traces.jsonl      # 事件轨迹（record_traces: true 时）
├─ plots/            # 图表与 HTML 报告（simlab plot / run --plot）
├─ ai_analysis.json  # AI 分析结构化结果（--analyze 时）
└─ ai_analysis.md    # AI 分析 Markdown 报告（--analyze 时）
```

九、AI 分析层（OpenAI / DeepSeek 双路径）

分析层（`ai.py`）接收 KPI 汇总，输出经 Pydantic 校验的 `KPIAnalysis`（执行摘要、关键发现、建议、注意事项），支持自定义业务问题。

路径	条件	机制
OpenAI Responses API	base_url 为空（默认）	<code>responses.parse(text_format=Pydantic模型)</code> 结构化输出
DeepSeek 兼容路径	base_url 非空	chat/completions + <code>json_object</code> 模式，Schema 嵌入提示词，返回后本地 Pydantic 二次校验

DeepSeek 使用要点：① DeepSeek 不支持 Responses API，必须设置 base_url 走兼容路径；② 模型名用 `deepseek-chat`；③ 密钥读 DEEPSEEK_API_KEY（自动推断）；④ AI 分析是决策辅助——KPI 数值由本地代码计算，模型只能解释提供的数据。

十、安全闭环控制 API

这一层实现“下一轮实验控制”，不是热修改正在运行的 SimPy 生成器：

不可变 ProjectConfig 快照 → 运行 SimPy 实验 → KPI 汇总 → AI 结构化提案（DeepSeek/OpenAI）
→ 人工审批 → 拒绝：审计并结束；批准：本地 allowlist + 完整配置校验 → 新配置版本 → 回到起点

启动

```
$env:SIMLAB_API_TOKEN = "替换为随机长字符串"
$env:SIMLAB_API_OUTPUT_ROOT = "outputs/api_sessions"
.\.venv\Scripts\simlab.exe serve --host 127.0.0.1 --port 8000
# 打开 http://127.0.0.1:8000/docs 使用 OpenAPI 交互页面
```

REST 路由

方法	路由	说明
POST	/v1/sessions	创建配置会话
GET	/v1/sessions/{id}	读取会话（版本号与配置哈希）
GET	/v1/sessions/{id}/allowed-actions	读取当前精确动作白名单
POST	/v1/sessions/{id}/proposals	人工/可信系统提交结构化提案
POST	/v1/sessions/{id}/proposals:generate	AI 根据最近 KPI 汇总生成待审批提案计划
POST	/v1/sessions/{id}/proposals/{pid}:approve	人工批准并应用到下一版配置
POST	/v1/sessions/{id}/proposals/{pid}:reject	人工拒绝
POST	/v1/sessions/{id}/runs	以当前配置快照运行实验
GET	/v1/sessions/{id}/runs/{rid}	读取运行记录（?include_result=true 带结果）
GET	/v1/sessions/{id}/audit	审批审计 + 服务事件
WS	/v1/sessions/{id}/events	事件流（Bearer 头，按 sequence 订阅/重放）

每个写操作要求唯一 operation_id 与当前 expected_version：重复同一操作返回原结果（幂等）；复用 ID 发不同内容或使用旧版本返回 409 冲突。服务端忽略请求中的 experiment.output_dir，结果写入服务端分配的 输出根目录/会话ID/运行ID/。

AI 动作安全边界（五道防线）

- 1. 动作类型与 target 必须逐字命中本地 allowlist。
- 2. requires_approval 必须为 true。
- 3. 动作应用到配置副本后必须重新通过完整 ProjectConfig 校验；参数网格场景逐项受服务端工作量上限约束。
- 4. 提案的配置哈希必须与当前版本一致，旧提案不能套用到新配置（409 过期提案）。
- 5. 模型不能设置 API key、模型名、输出路径、随机 seed 或任意 dotted path；没有 shell/文件/eval/exec 工具。多动作计划中 enable_resource 必须排在所有带下标工位参数动作之后（防下标漂移）。

十一、MCP 接口

安装 [mcp] 后启动 stdio 服务：simlab-mcp.exe（依赖 SIMLAB_API_URL / SIMLAB_API_TOKEN）。MCP 进程通过 REST 连接正在运行的服务，两端读取同一会话、版本与审计。可用工具仅 validate_project / get_session / get_allowed_actions / submit_proposal / list_events，不暴露建会话、运行、批准、拒绝——模型无法用“另建会话再运行”绕过审批边界。

十二、事件轨迹记录与 XES 导出 (P0)

`trace.py`：开启 `experiment.record_traces: true` 后，仿真内核逐实体记录事件轨迹，保存结果时导出 `traces.xes` 与 `traces.jsonl`。

事件	activity	lifecycle	附加属性
到达	arrival	complete	—
工位开始服务	工位名	start	wait_time
工位服务结束	工位名	complete	service_time
完成	completion	complete	—

case id 形如 `{scenario}:r{replication}:{customer_id}`（含场景名，避免参数网格下跨场景实体混入同一 case）；org:resource 为工位名；`simlab:time` 保留原始浮点仿真时间。XES 时间映射：仿真时间当作业务时间单位，以 1970-01-01T00:00:00Z 为基准加偏移形成 `time:timestamp`。

十三、数据采集与模型挖掘 (simlab mine)

`mining.py` 对应目标框架的 Data collection / Data preparation / Input data 环节：从事件日志估计模型配置，实现**数据驱动模型生成**。刻意不依赖 pm4py (AGPL)，XES 解析手写实现。

挖掘内容

- **工位序列**：每条轨迹的工位活动首次出现顺序做多数投票（v1 仅支持串行拓扑，分支/循环会以警告呈现）。
- **到达间隔 / 服务时间分布**：按变异系数启发式拟合—— $cv \geq 0.9$ 指数、 $cv \leq 0.2$ 定值、其余三角（low/mode/high = 最小/中位/最大）。
- **工位并发容量**：start→complete 区间重叠扫描的最大并发数；多场景日志（case id 含 `:r`）按场景组分别计算后取众数（平票取最小值）。
- **周期时间目标**：case 周期时间的 P80；**until/warmup**：最大完成时间 / 其 10%。

用法

```
simlab mine outputs/service_center/traces.xes --output mined.yaml --project-name mined
# 也支持 JSONL: simlab mine real_system.jsonl --output mined.yaml
# 生成后用 simlab validate 校验、simlab run 验证后再投入使用
```

诚实边界：分布拟合是启发式，小样本/多场景合并的估计仅供起点；组间参数差异超过 20% 时会显示警告。真实系统日志需要符合“每条 case 含到达 + 各工位 start/complete 事件”的事件模型（外部日志可通过预处理转换）。

十四、LLM 蓝图式模型设计 (simlab design)

`design.py` 对应目标框架的 Model generation (Blueprint model instructions) 环节：把自然语言需求描述转成**完整仿真配置蓝图**。与 `agent.py` 的差异：`design` 生成“从零开始的模型结构 + 参数”，`agent` 只对已有模型做白名单内参数调适。

```
simlab design "设计一个体检中心：前台登记平均3分钟（指数分布），医生检查三角分布
4/8/15分钟，每小时12人到达，营业8小时，2名医生，服务水平目标30分钟" --output clinic.yaml
# 生成后: simlab validate clinic.yaml → simlab run clinic.yaml
```

安全边界：模型只输出 `SimulationConfig` 结构化数据（经本地 `Pydantic` 严格校验后才包装成 `ProjectConfig`），绝不输出代码；生成的配置必须人工 `validate/run` 验证后投入使用。双提供商路径与 `ai.py/agent.py` 完全一致；DeepSeek 端点自动用 `deepseek-chat`。

十五、结果可视化 (simlab plot)

`viz.py` 对应 Model visualization / Model comparison / Decision support 环节（需 `pip install -e ".[viz]"`）。配色遵循数据可视化规范：验证过的固定顺序分类调色板、浅色表面、细标记、直接标注、弱化轴线，每图单一量纲（无双轴）。

```
simlab plot outputs/service_center/results.json --output outputs/service_center/plots
# 或仿真后一步到位: simlab run examples/service_center.yaml --plot
```

产物	内容
<code>kpi_comparison.png</code>	关键 KPI 场景对比（小倍图，误差棒为 95% CI）
<code>station_utilization.png</code>	各工位利用率按场景分组
<code>traces_gantt.png</code>	前 N 个实体的轨迹甘特图（需 <code>record_traces</code> ）
<code>model_structure.png</code>	模型结构图（到达 → 工位 → 完成）
<code>report.html</code>	KPI 汇总表 + 全部图表的 HTML 报告

十六、模块清单与职责

模块	职责
<code>config.py</code>	YAML/JSON 配置模型与严格校验；环境变量覆盖
<code>rng.py</code>	BLAKE2b 命名空间 seed 派生
<code>simulation.py</code>	SimPy 单次 replication：到达流、串行工位、资源请求；可选轨迹记录
<code>kpi.py</code>	KPI 收集器（时间面积法）与指标目录
<code>trace.py</code>	事件轨迹记录与 XES / JSONL 导出（P0）
<code>mining.py</code>	事件日志读取（XES/JSONL）与模型配置挖掘
<code>experiment.py</code>	场景展开、多进程调度、统计汇总、结果落盘
<code>ai.py</code>	KPI 分析适配器（双路径）
<code>agent.py</code>	动作提案适配器（双路径），只出提案不执行
<code>design.py</code>	LLM 蓝图式模型设计（双路径）
<code>policy.py</code>	白名单目录、动作应用、工作量上限、配置哈希
<code>workflow.py</code>	版本化审批状态机、审计日志、幂等操作
<code>service.py</code>	会话编排：提案→审批→不可变配置快照运行
<code>api.py</code>	FastAPI REST + WebSocket 控制面
<code>mcp_server.py</code>	MCP v2 受限客户端（提案-only 门面）
<code>viz.py</code>	结果图表与 HTML 报告（matplotlib 可选依赖）
<code>cli.py</code>	命令行入口：validate / run / analyze / plot / mine / design / serve

十七、扩展指南

- 改模型拓扑**：在 `simulation.py` 的 `customer()` 加条件路由、返工、优先级或批处理。
- 加业务参数**：在 `config.py` 新增字段（保持 `extra="forbid"`），并在 `policy.py` 决定是否暴露给 AI。
- 加 KPI**：在 `KPICollector` 通过事件记录新增指标并登记到指标目录；不要让 AI 参与 KPI 数值计算。
- 加随机过程**：延续“独立随机流”规则（按名称派生 seed）。
- 定制模型**：增加新的 runner，而不是把业务规则硬编码进 CLI。
- 重型流程挖掘**：需要 Petri 网/过程树、一致性校验时，通过可选 `extra` 接入 `pm4py`（AGPL：只做文件交互，不并入 MIT 核心）。
- 挖掘能力扩展**：当前 `mine` 仅支持串行拓扑与三种分布启发式；可扩展分支/并行发现（P2 拓扑泛化）与更多分布拟合方法。

十八、已知限制

- 会话、幂等记录与事件日志在单进程内存中，服务重启即丢失；生产应换 SQLite/PostgreSQL。
- WebSocket broker 面向单 Uvicorn worker；多实例需 Redis 或消息总线。
- 已运行的 replication 不会暂停/序列化/热改；批准动作只影响下一次运行（刻意的审计特性）。
- API 暂无角色型 JWT；SIMLAB_API_TOKEN 是单一管理员 Bearer Token。
- 置信区间使用正态近似；replication 少或重尾明显时建议增加次数或改用 bootstrap / t 区间。
- DeepSeek 兼容路径的 JSON 模式不保证 schema（本地 Pydantic 校验兜底）。
- mine 仅支持串行拓扑与启发式分布拟合；多场景合并估计需人工复核。
- 可视化输出为静态 PNG/HTML（无交互悬浮层），适合报告场景；交互式图表可后续用 Plotly 等扩展。

十九、本机环境备忘

- Python: F:\PY\Python (3.13.0, 64 位)；虚拟环境 .venv 已修复指向该解释器。
- 已安装: simlab 0.2.0 (editable)、simpy 4.1.2、openai 2.54.0、pydantic 2.13.4、fastapi、uvicorn、httpx、matplotlib 3.11.1、pytest、ruff。
- 密钥: .env 中已配置 DEEPSEEK_API_KEY 与 SIMLAB_OPENAI_BASE_URL=https://api.deepseek.com；模型 deepseek-chat。Git Bash 中用 `set -a && . ./env && set +a` 加载。
- 测试: `./venv/Scripts/python.exe -m pytest -q` (95 项)；代码风格: `./venv/Scripts/ruff.exe check src tests`。
- 闭环演示: `closed_loop_demo.py` (`--approve / --reject / --audit`)，配合 simlab serve 使用。